Nama : Hibrizi Damas Falah
NIM : 24250009
Prodi : Sistem Informasi

Aplikasi Berbasis Service
OAuth 2.0 dan OpenID Connect — Authorization and Authentication

A. Pengertian

OAuth 2.0 adalah kerangka otorisasi yang dipublikasikan IETF dalam RFC 6749 pada tahun 2012. Standar ini memungkinkan sebuah aplikasi memperoleh akses terbatas ke sumber daya milik pengguna tanpa pernah menerima kata sandi pengguna tersebut. Akses diberikan dalam bentuk *access token* yang memiliki cakupan (*scope*) dan masa berlaku tertentu.

OpenID Connect (OIDC) adalah lapisan identitas yang dibangun di atas OAuth 2.0 dan dirilis OpenID Foundation pada tahun 2014. OIDC menambahkan kemampuan autentikasi, yaitu cara bagi aplikasi untuk memastikan siapa pengguna yang sedang masuk dan memperoleh informasi dasar tentang pengguna itu melalui *ID Token*.

Secara singkat, OAuth 2.0 menjawab pertanyaan “apa yang boleh diakses aplikasi ini”, sedangkan OpenID Connect menjawab pertanyaan “siapa pengguna yang sedang masuk”. Keduanya banyak dipakai pada fitur “Masuk dengan Google” dan pada layanan yang memisahkan penyedia identitas dari aplikasi.

B. Peran dalam OAuth 2.0

OAuth 2.0 mendefinisikan empat peran yang saling berinteraksi dalam proses pemberian akses.

Peran	Penjelasan	Contoh pada sistem absensi
Resource Owner	Pemilik data yang dapat memberikan izin akses.	Guru atau orang tua sebagai pemilik akun.
Client	Aplikasi yang meminta akses atas nama pengguna.	Aplikasi absensi Android.
Authorization Server	Server yang mengautentikasi pengguna dan menerbitkan token.	Server identitas sekolah atau penyedia identitas pihak ketiga.
Resource Server	Server yang menyimpan data dan menerima token untuk memberi akses.	API data kehadiran siswa.

C. Alur Authorization Code dengan PKCE

Alur yang direkomendasikan untuk aplikasi mobile adalah Authorization Code Flow yang dilengkapi PKCE (Proof Key for Code Exchange, RFC 7636). PKCE melindungi kode otorisasi dari pencurian pada aplikasi yang tidak dapat menyimpan rahasia dengan aman. Tahapannya sebagai berikut.

1. Aplikasi membuat code_verifier acak beserta code_challenge turunannya, lalu mengarahkan pengguna ke Authorization Server.
2. Pengguna melakukan login dan menyetujui izin yang diminta aplikasi.
3. Authorization Server mengembalikan authorization code ke aplikasi melalui redirect URI.
4. Aplikasi menukar kode tersebut dengan token sambil mengirim code_verifier sebagai bukti.
5. Authorization Server menerbitkan access token, dan ID Token apabila scope openid diminta.
6. Aplikasi memanggil API dengan access token pada header Authorization, lalu Resource Server memvalidasi token sebelum memberikan data.

Alur Implicit dan Resource Owner Password Credentials tidak lagi dianjurkan karena rentan terhadap kebocoran token dan kredensial. Panduan keamanan terbaru dari IETF (RFC 9700, 2025) menegaskan penggunaan Authorization Code dengan PKCE.

D. Komponen OpenID Connect

ID Token berbentuk JSON Web Token (JWT) yang ditandatangani dan memuat klaim tentang proses autentikasi, antara lain *iss* (penerbit), *sub* (identitas unik pengguna), *aud* (aplikasi tujuan), *exp* (waktu kedaluwarsa), dan *iat* (waktu penerbitan).

Scope openid wajib disertakan dalam permintaan agar sebuah alur OAuth 2.0 diperlakukan sebagai permintaan OIDC. Scope tambahan seperti *profile* dan *email* digunakan untuk meminta data pengguna yang lebih lengkap.

UserInfo Endpoint adalah titik akses yang mengembalikan informasi profil pengguna ketika dipanggil dengan access token. **Discovery Document** pada alamat /.well-known/openid-

configuration memuat daftar endpoint dan konfigurasi server sehingga aplikasi dapat menyesuaikan diri secara otomatis.

E. Perbedaan OAuth 2.0 dan OpenID Connect

Aspek	OAuth 2.0	OpenID Connect
Fungsi utama	Otorisasi (pemberian izin akses).	Autentikasi (verifikasi identitas pengguna).
Token utama	Access token.	ID Token, di samping access token.
Format token	Tidak ditentukan; dapat opaque atau JWT.	ID Token wajib berformat JWT.
Informasi pengguna	Tidak distandarkan.	Distandarkan melalui klaim dan UserInfo Endpoint.
Scope khusus	Ditentukan oleh masing-masing layanan.	openid, profile, email, dan sejenisnya.

Access token tidak boleh dipakai sebagai bukti identitas pengguna. Untuk keperluan login, aplikasi harus memvalidasi ID Token.

F. Keterkaitan dengan SOA

Pada arsitektur berbasis layanan, aplikasi dipecah menjadi banyak layanan yang saling memanggil. Tanpa mekanisme yang seragam, setiap layanan harus mengelola login dan hak aksesnya sendiri. OAuth 2.0 dan OpenID Connect memusatkan proses tersebut pada satu Authorization Server sehingga layanan lain cukup memvalidasi token.

Aspek SOA	Penerapan	Penjelasan sederhana
Autentikasi pengguna	OpenID Connect dan ID Token	Identitas pengguna diverifikasi sekali dan dapat dipakai lintas layanan (single sign-on).
Otorisasi antarlayanan	Access token dan scope	Setiap layanan hanya menerima permintaan yang membawa token dengan scope yang sesuai.
Komunikasi layanan ke layanan	Client Credentials Flow	Layanan memperoleh token atas nama dirinya sendiri tanpa keterlibatan pengguna.
Keamanan transport	TLS dan token bertanda tangan	Token dikirim melalui koneksi terenkripsi dan keasliannya dapat diperiksa.

G. Contoh Penerapan pada Sistem Informasi Absensi Android

Sebagai ilustrasi, sistem informasi absensi berbasis Android untuk monitoring kehadiran siswa di SMP Negeri Marijoa melayani empat aktor, yaitu siswa, guru, admin, dan orang tua. Aplikasi Android bertindak sebagai client, server identitas sebagai Authorization Server, dan API data kehadiran sebagai Resource Server. Ketika pengguna masuk, OpenID Connect memastikan identitasnya dan ID Token memuat peran pengguna, sedangkan OAuth 2.0 membatasi data yang dapat diakses melalui scope pada access token.

Aktor	Hak akses	Contoh scope
Siswa	Melihat riwayat kehadiran pribadi.	absensi.read.self
Guru	Mencatat dan melihat kehadiran siswa di kelasnya.	absensi.read, absensi.write
Admin	Mengelola data siswa, kelas, dan laporan.	absensi.admin
Orang tua	Memantau kehadiran anaknya saja.	absensi.read.child

Dengan pola ini, orang tua yang masuk tidak dapat membuka data siswa lain meskipun menggunakan API yang sama, karena Resource Server memeriksa scope dan identitas pada token sebelum menjawab permintaan. Nama scope di atas hanyalah contoh dan dapat disesuaikan dengan rancangan sistem.

H. Manfaat

Kata sandi tidak dibagikan ke aplikasi. Pengguna cukup login pada Authorization Server, sehingga risiko kebocoran kredensial berkurang.

Akses dapat dibatasi dan dicabut. Scope dan masa berlaku token membatasi cakupan akses, dan token dapat dicabut tanpa mengganti kata sandi.

Login terpusat. Satu identitas dapat digunakan pada banyak layanan sehingga pengelolaan akun lebih sederhana dan pengalaman pengguna lebih baik.

Standar terbuka. Banyak pustaka dan penyedia identitas mendukung kedua standar ini sehingga integrasi antarplatform lebih mudah.

I. Tantangan

Konfigurasi yang rumit. Kesalahan pada redirect URI, validasi token, atau penentuan scope dapat membuka celah keamanan.

Ketergantungan pada Authorization Server. Gangguan pada server ini menyebabkan seluruh layanan yang bergantung padanya tidak dapat diakses.

Penyimpanan dan masa berlaku token. Token yang bocor dapat disalahgunakan selama masih berlaku, sehingga perlu dikombinasikan dengan masa berlaku singkat, refresh token, dan penyimpanan yang aman pada perangkat.

Kesalahpahaman fungsi. OAuth 2.0 sering dianggap sebagai protokol login, padahal autentikasi hanya disediakan oleh OpenID Connect.

J. Kesimpulan

OAuth 2.0 menyediakan mekanisme otorisasi yang aman dan terbatas, sedangkan OpenID Connect melengkapinya dengan autentikasi berbasis ID Token. Pada aplikasi berbasis service, keduanya memungkinkan identitas dan hak akses dikelola secara terpusat sehingga setiap layanan cukup memvalidasi token yang diterima. Keamanan penerapannya tetap bergantung pada konfigurasi yang benar, penggunaan Authorization Code dengan PKCE, dan pengelolaan token yang disiplin.

Referensi

1. Hardt, D. (Ed.). (2012). The OAuth 2.0 Authorization Framework (RFC 6749). Internet Engineering Task Force.
2. Jones, M., & Hardt, D. (2012). The OAuth 2.0 Authorization Framework: Bearer Token Usage (RFC 6750). Internet Engineering Task Force.
3. Sakimura, N., Bradley, J., & Agarwal, N. (2015). Proof Key for Code Exchange by OAuth Public Clients (RFC 7636). Internet Engineering Task Force.
4. Sakimura, N., Bradley, J., Jones, M., de Medeiros, B., & Mortimore, C. (2014). OpenID Connect Core 1.0. OpenID Foundation.
5. Lodderstedt, T., Bradley, J., Labunets, A., & Fett, D. (2025). Best Current Practice for OAuth 2.0 Security (RFC 9700). Internet Engineering Task Force.